

Sales Performance & Forecasting Analysis

Python & Data Science Internship Assessment

Project Objective

This project performs an end-to-end analysis of the Superstore retail sales dataset. The analysis covers data cleaning, exploratory data analysis, statistical analysis, and machine-learning-based sales forecasting.

Technologies Used

Python 3, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn and Jupyter Notebook.

Dataset

Superstore Sales Dataset — Kaggle.

1. Project Description

In this project, I analyzed the Superstore sales data to understand sales patterns and tried to predict monthly sales using machine learning. The workflow includes inspection and cleaning of the raw dataset, exploratory visualizations, statistical summaries, predictive modeling, model evaluation, and business recommendations.

2. Data Cleaning & Preparation

The dataset contains 9,800 rows and 18 columns. The data was inspected for missing values, duplicate records, and incorrect data types.

There were 11 missing values in the Postal Code column. These values were replaced with the label 'Unknown' because Postal Code is a location identifier and replacing it with a numerical average would not be meaningful.

The Order Date and Ship Date columns were converted from text to datetime format. After conversion, both date columns contained zero missing values. No duplicate rows were found in the dataset.

3. Statistical Summary

Statistic	Sales
Mean	230.77
Median	54.49
Standard Deviation	626.65
Minimum	0.444
Maximum	22,638.48

Interpretation: The mean sales value of 230.77 is substantially higher than the median of 54.49. This indicates that a relatively small number of high-value orders strongly influence the average sales value.

4. Code and Output Screenshots

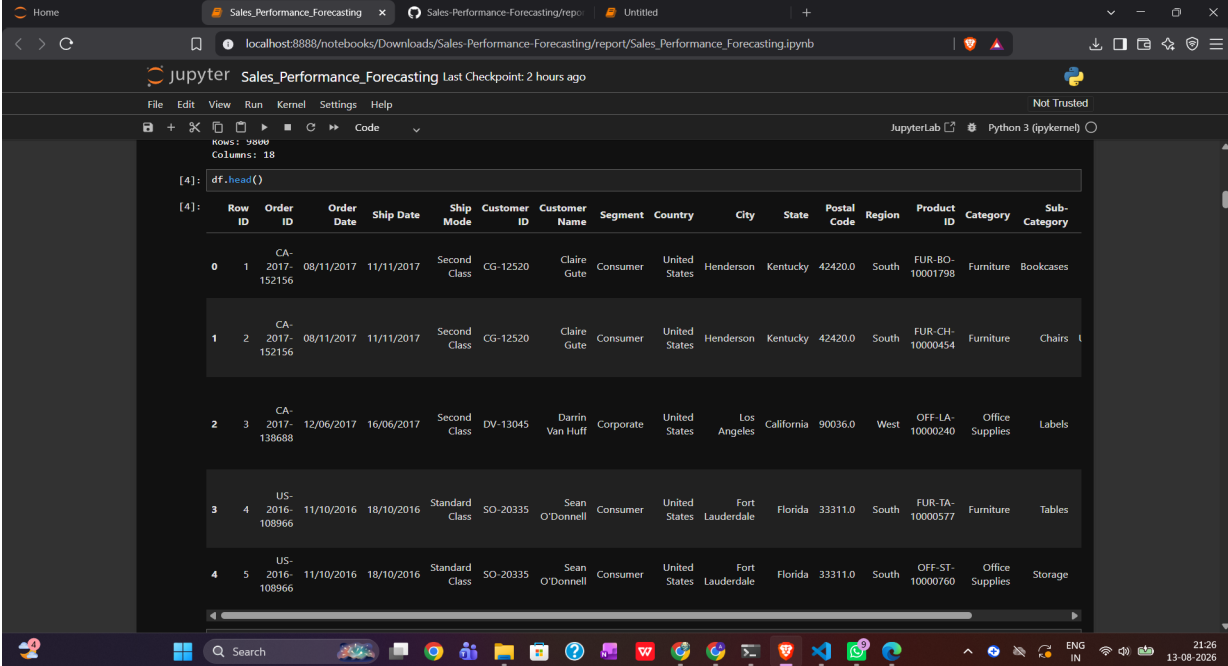


Figure 1 displays a JupyterLab interface showing a dataset preview. The code cell [4] contains the command `df.head()`. The output shows the first 5 rows of the dataset, which has 18 columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, and Sales.

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-BO-10001798	Furniture	Bookcases
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-CH-10000454	Furniture	Chairs
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036.0	West	OFF-LA-10000240	Office Supplies	Labels
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	FUR-TA-10000577	Furniture	Tables
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311.0	South	OFF-ST-10000760	Office Supplies	Storage

Figure 1 — Dataset Preview

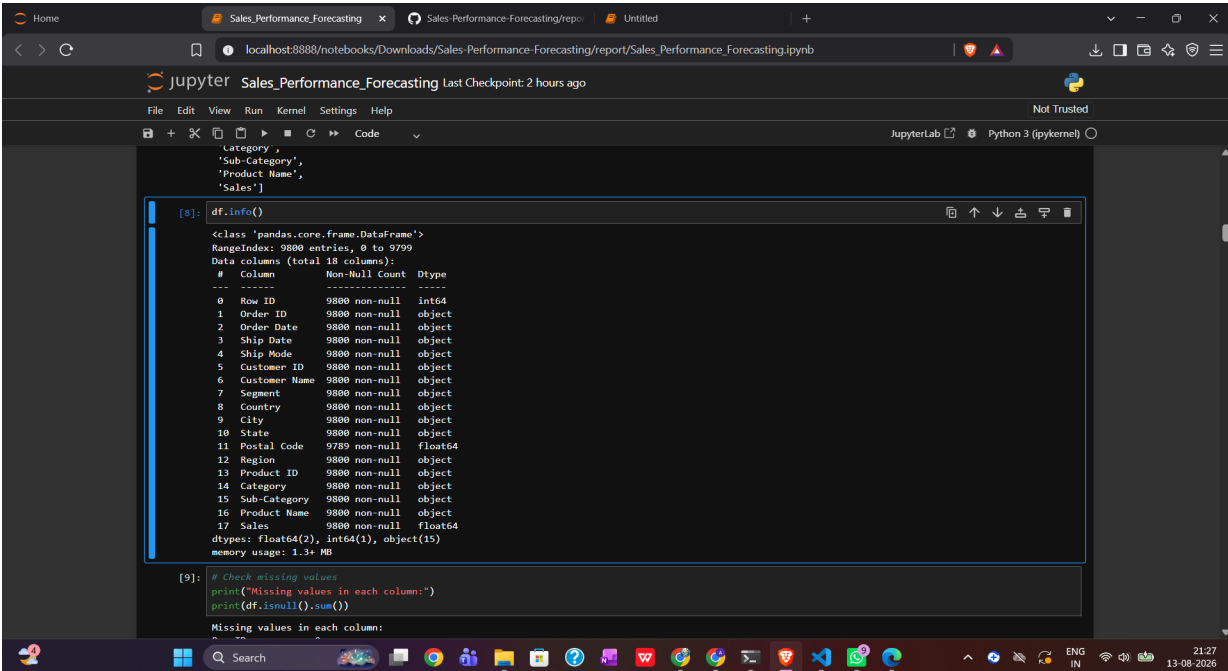


Figure 2 displays a JupyterLab interface showing dataset information. The code cell [8] contains the command `df.info()`. The output shows the data types and non-null counts for each column. The code cell [9] contains a check for missing values.

```
[8]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
 #   Column             Non-Null Count  Dtype
---  -
 0   Row ID             9800 non-null  int64
 1   Order ID           9800 non-null  object
 2   Order Date         9800 non-null  object
 3   Ship Date          9800 non-null  object
 4   Ship Mode          9800 non-null  object
 5   Customer ID        9800 non-null  object
 6   Customer Name      9800 non-null  object
 7   Segment            9800 non-null  object
 8   Country            9800 non-null  object
 9   City               9800 non-null  object
10  State              9800 non-null  object
11  Postal Code        9789 non-null  float64
12  Region            9800 non-null  object
13  Product ID         9800 non-null  object
14  Category           9800 non-null  object
15  Sub-Category       9800 non-null  object
16  Product Name       9800 non-null  object
17  Sales              9800 non-null  float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB

[9]: # Check missing values
print("Missing values in each column:")
print(df.isnull().sum())

Missing values in each column:
```

Figure 2 — Dataset Information

The JupyterLab interface displays a notebook titled 'Sales_Performance_Forecasting'. The code in cell [9] checks for missing values in each column. The output shows that all columns have 0 missing values. Cell [10] checks for duplicate rows, and the output shows 0 duplicate rows. Cell [11] checks the data types of the columns, showing that 'Row ID' is 'int64' and 'Order ID' is 'object'.

```
[9]: # Check missing values
print("Missing values in each column:")
print(df.isnull().sum())

Missing values in each column:
Row ID      0
Order ID    0
Order Date  0
Ship Date   0
Ship Mode   0
Customer ID  0
Customer Name 0
Segment    0
Country     0
City        0
State       0
Postal Code 11
Region      0
Product ID  0
Category    0
Sub-Category 0
Product Name 0
Sales       0
dtype: int64

[10]: # Check duplicate rows
print("Number of duplicate rows:", df.duplicated().sum())

Number of duplicate rows: 0

[11]: # Check data types
print("Data types:")
print(df.dtypes)

Data types:
Row ID      int64
Order ID    object
```

Figure 3 — Missing Value Check

The JupyterLab interface displays a notebook titled 'Sales_Performance_Forecasting'. Cell [12] shows the statistical summary of the dataset. Cell [22] handles missing 'Postal Code' values by filling them with 'Unknown'. Cell [23] checks for missing 'Postal Code' values after cleaning, showing 0 missing values. Cell [24] converts 'Order Date' and 'Ship Date' columns to datetime format.

```
[12]: # statistical summary
df.describe()

[12]:
```

	Row ID	Postal Code	Sales
count	9800.000000	9789.000000	9800.000000
mean	4900.500000	55273.322403	230.769059
std	2829.160653	32041.223413	626.651875
min	1.000000	1040.000000	0.444000
25%	2450.750000	23223.000000	17.248000
50%	4900.500000	58103.000000	54.490000
75%	7350.250000	90008.000000	210.605000
max	9800.000000	99301.000000	22638.480000

```
[22]: # Handle missing Postal Codes
df["Postal Code"] = df["Postal Code"].fillna("Unknown")

[23]: print("Missing Postal Code values after cleaning:",
df["Postal Code"].isnull().sum())

Missing Postal Code values after cleaning: 0

[24]: # Convert date columns to datetime
df["Order Date"] = pd.to_datetime(df["Order Date"], dayfirst=True)
df["Ship Date"] = pd.to_datetime(df["Ship Date"], dayfirst=True)
```

Figure 4 — Statistical Summary

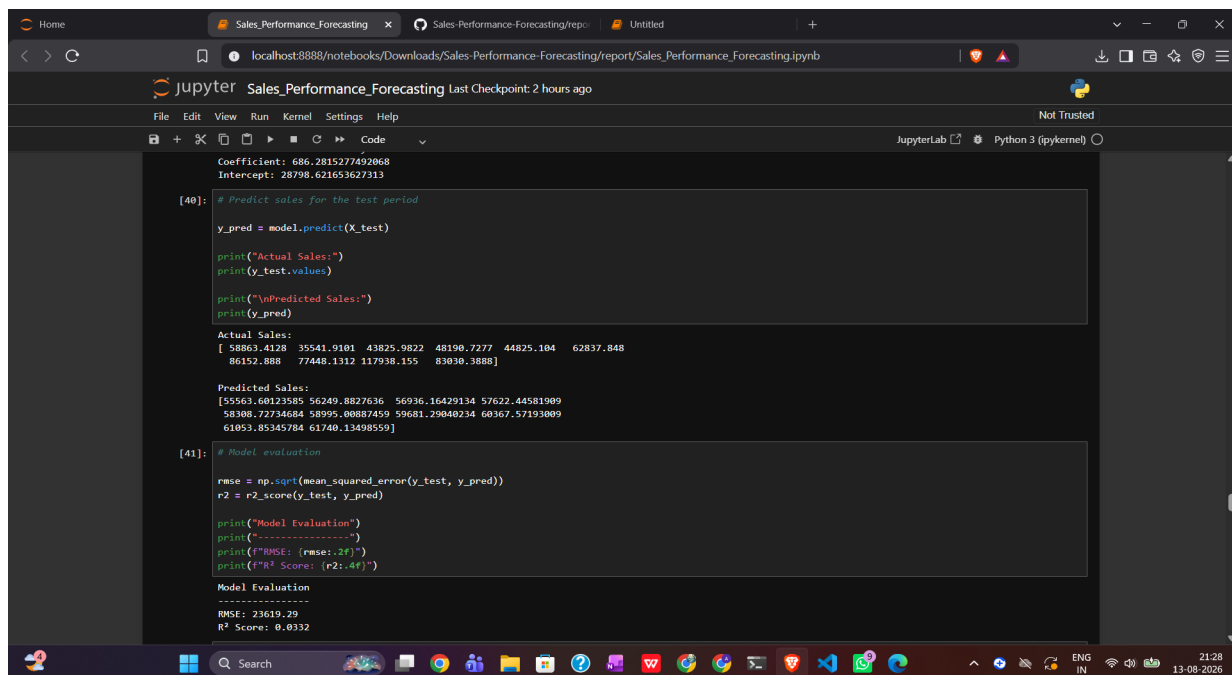


Figure 5 — Linear Regression Output

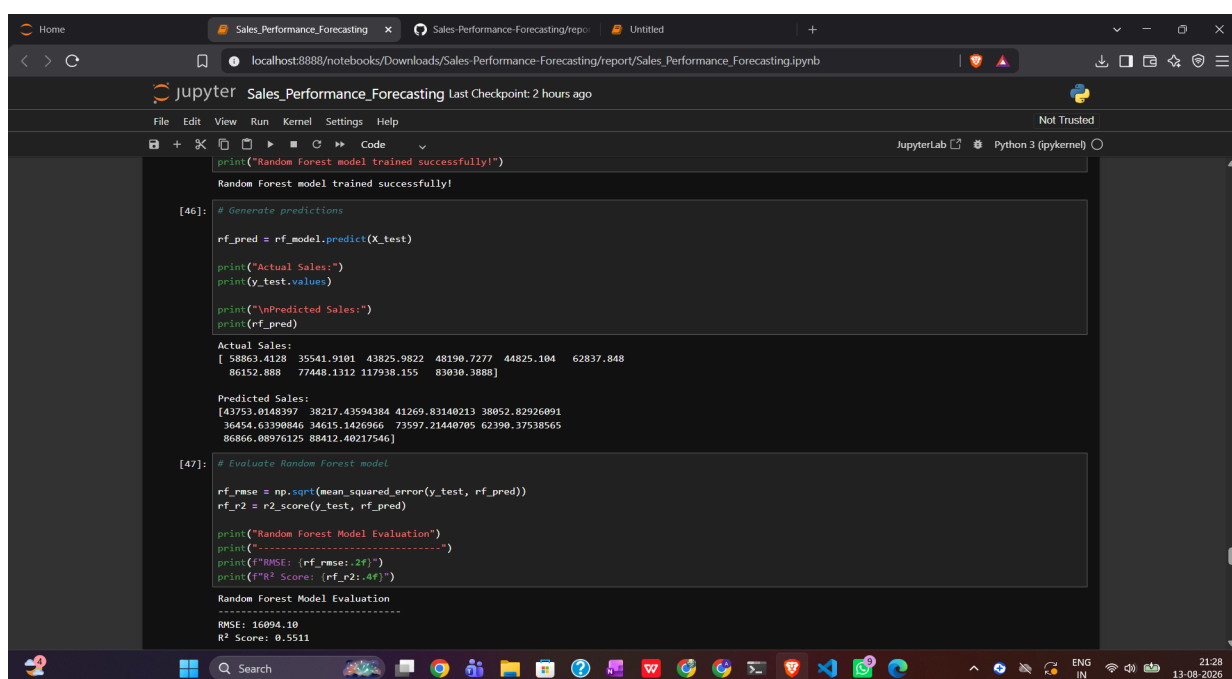


Figure 6 — Random Forest Output

4. Exploratory Data Analysis

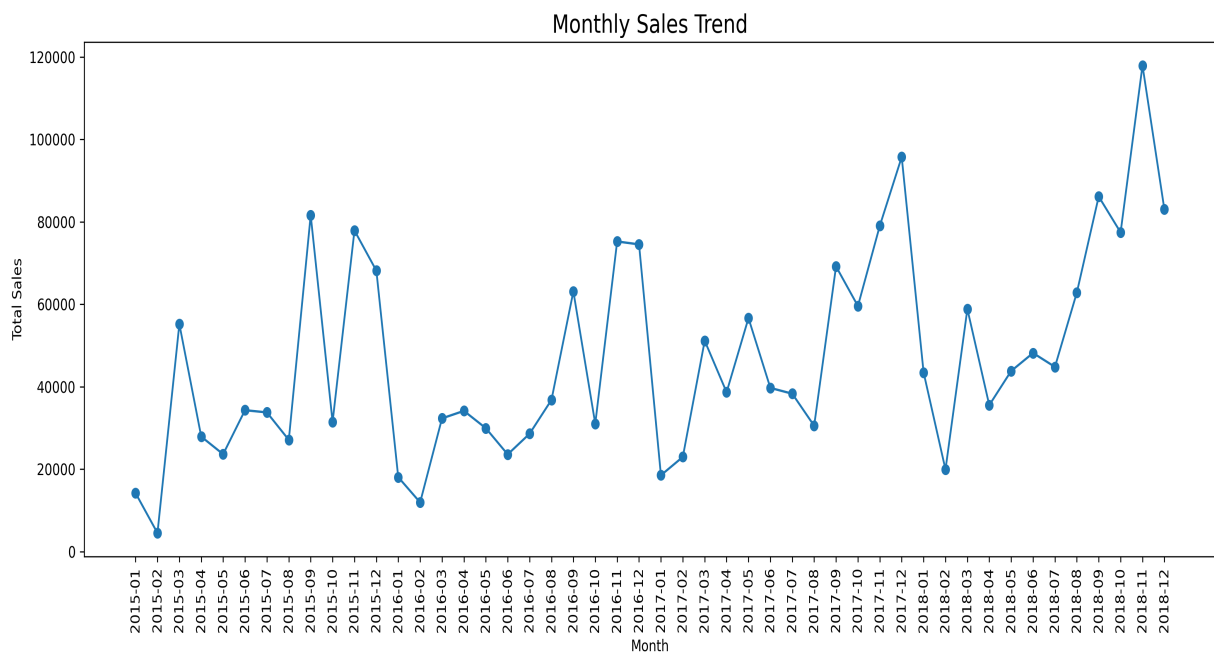


Figure 1 — Monthly Sales Trend

Interpretation: The monthly sales trend shows that sales change quite a lot from month to month. Some months have much higher sales than others, which shows that sales are not consistent throughout the year. November 2018 had the highest monthly sales, while February 2015 had the lowest.

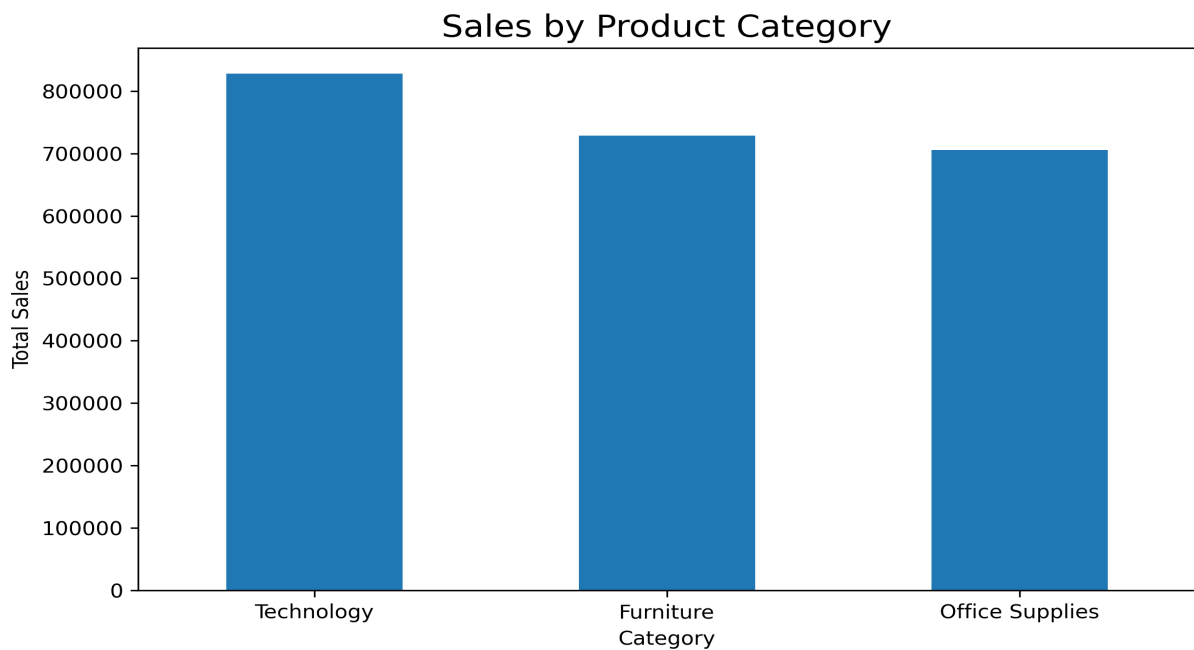


Figure 2 — Sales by Product Category

Interpretation: Technology is the highest-performing category with total sales of 827,455.87. Furniture and Office Supplies also contribute a large amount to total sales. This shows that Technology products are an important part of the business.

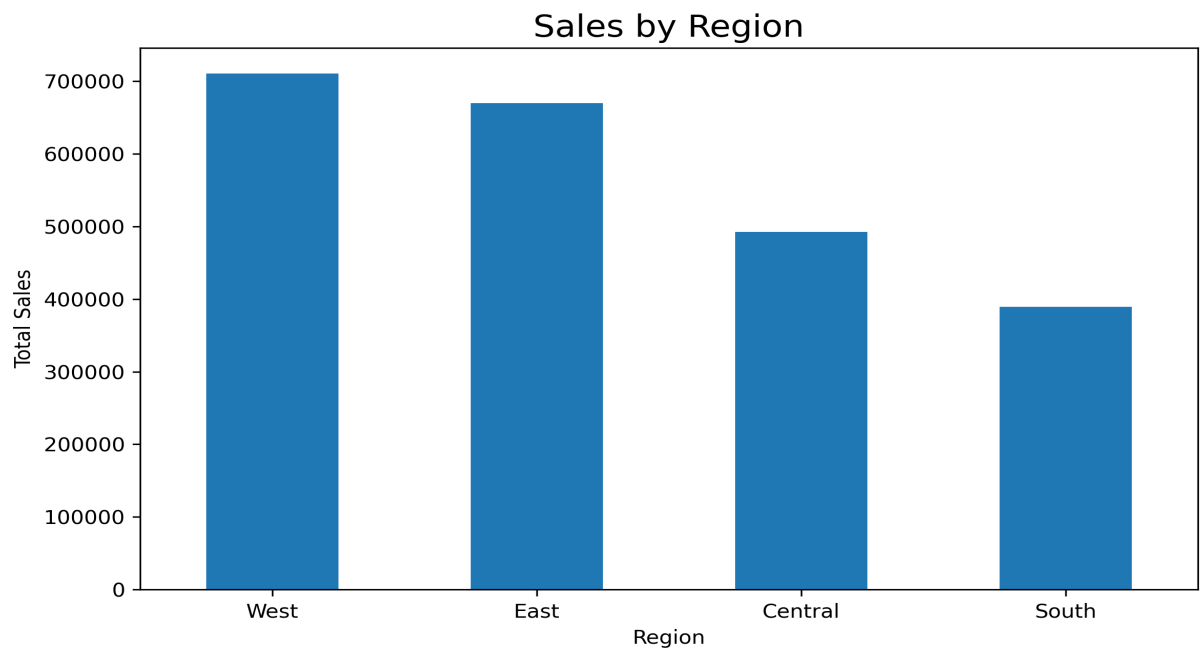


Figure 3 — Sales by Region

Interpretation: The West region has the highest total sales at 710,219.68. The performance of the regions is different, with some regions contributing much less than the West. The lower-performing regions could be studied further to understand the reasons for the difference.

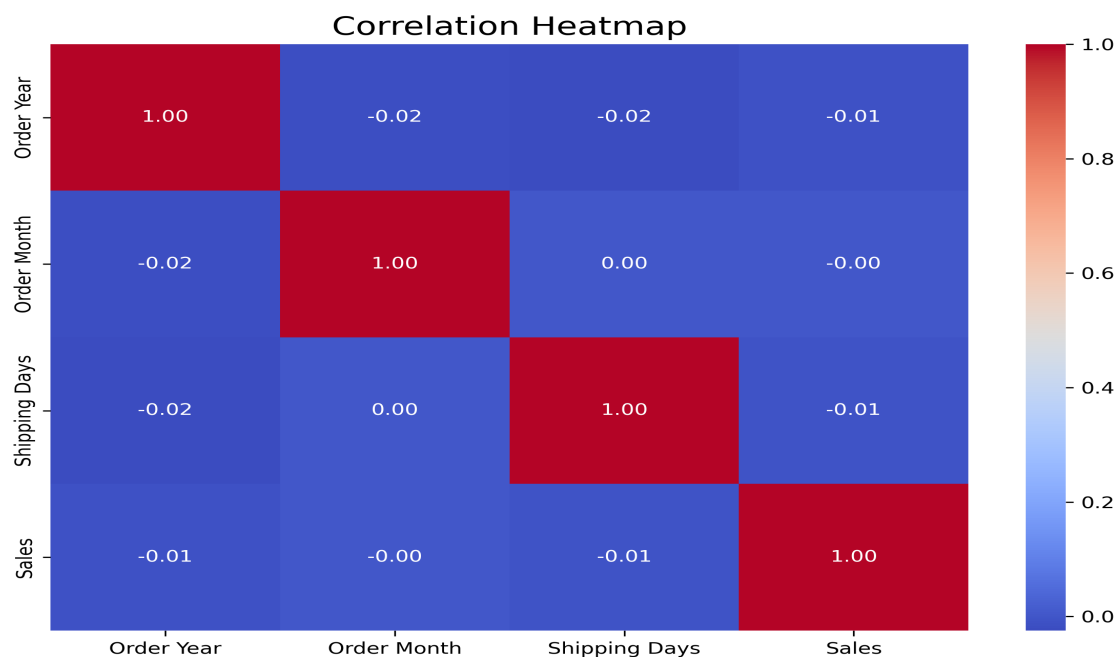


Figure 4 — Correlation Heatmap

Interpretation: The heatmap shows the relationships between the numerical variables in the dataset. It helps identify which variables have stronger or weaker relationships with Sales and Profit. These relationships can also help in deciding which variables may be useful for analysis and modelling.

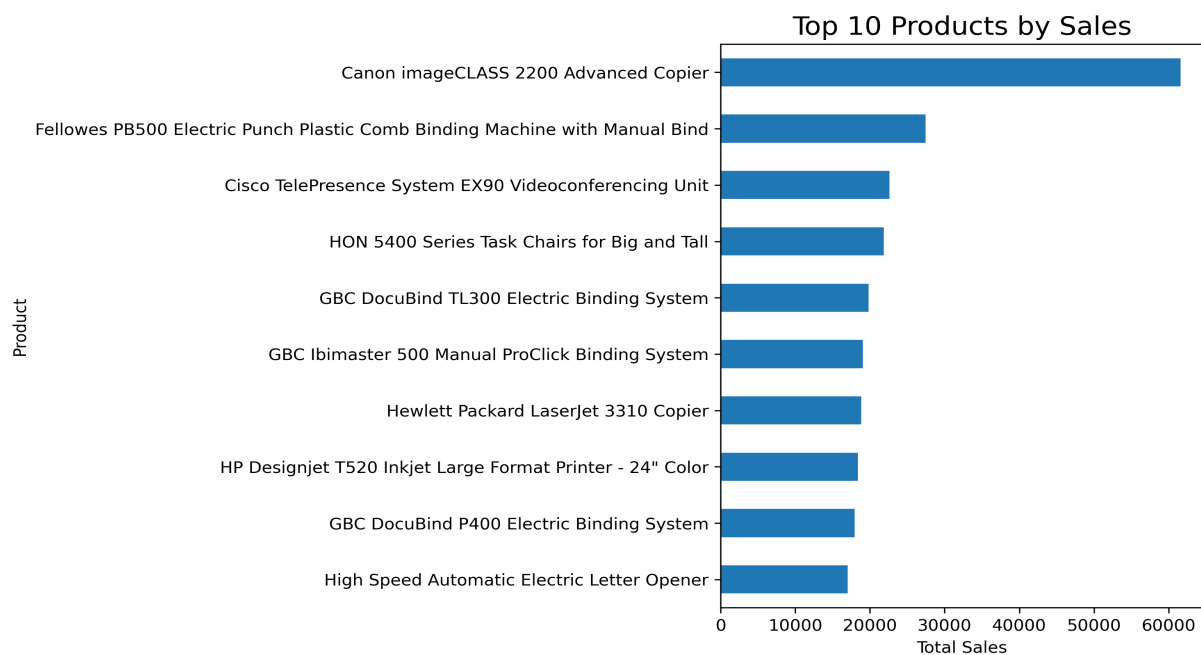


Figure 5 — Top 10 Products by Sales

Interpretation: The top 10 products contribute a significant amount of sales compared with other individual products. The Canon imageCLASS 2200 Advanced Copier is the top-selling product with total sales of 61,599.82. These high-performing products can be given more attention during inventory and sales planning.

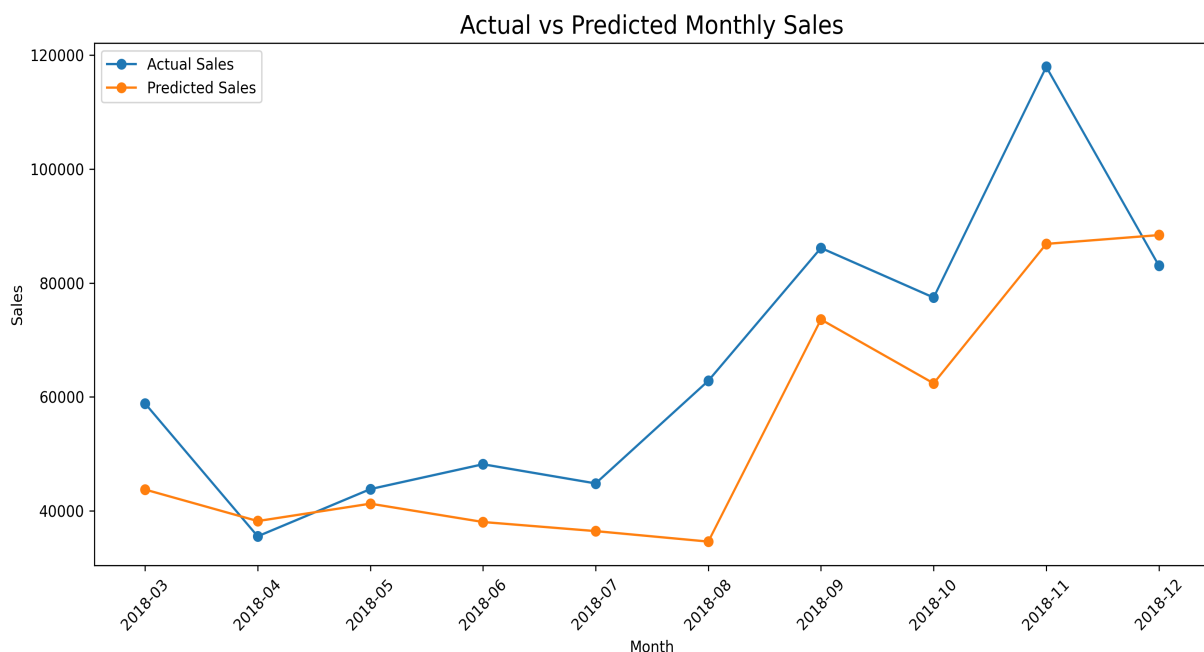


Figure 6 — Actual vs Predicted Monthly Sales

Interpretation: The chart compares the actual monthly sales with the values predicted by the model. The Random Forest predictions follow the general pattern of the actual sales better than the simple Linear Regression model. However, there are still differences between actual and predicted values, so the model can be improved further.

5. Key Insights

- **Technology performed the best:** Technology had the highest total sales of 827,455.87, followed by Furniture at 728,658.58 and Office Supplies at 705,422.33. This shows that Technology products contribute the most to overall sales.
- **Regional performance is different:** The West region had the highest sales at 710,219.68, while the South region had the lowest sales at 389,151.46. This difference shows that there is scope to improve sales performance in the lower-performing regions.
- **Sales change a lot over time:** Monthly sales ranged from 4,519.89 in February 2015 to 117,938.15 in November 2018. This shows that some months have much higher sales than others.
- **Top-selling product:** The Canon imageCLASS 2200 Advanced Copier was the highest-selling product, generating total sales of 61,599.82. High-performing products like this can be given more attention during inventory and sales planning.

6. Predictive Modeling

A Linear Regression model was first developed using monthly time sequence information. The model achieved an RMSE of 23,619.29 and an R^2 score of 0.0332. This shows that a simple linear trend was not able to capture the changes in monthly sales very well.

A Random Forest Regression model was then developed using time-based and seasonal features such as month number, year, month, and cyclical month features. The model achieved an RMSE of **16,094.10** and an R^2 score of **0.5511**.

Model	RMSE	R^2 Score
Linear Regression	23,619.29	0.0332
Random Forest	16,094.10	0.5511

Model Interpretation: Random Forest performed better than Linear Regression because it had a lower RMSE and a higher R^2 score. The R^2 value of 0.5511 means that the model was able to explain around 55% of the variation in the monthly sales data. However, there is still room for improvement in the forecasting results.

7. Business Recommendations

1. Focus more on Technology products for inventory and promotions because this category had the highest total sales.

2. Look into the lower sales of the South region and try to understand whether factors such as customer demand, product availability, pricing, or marketing are affecting its performance.
3. Give more attention to high-performing products such as the Canon imageCLASS 2200 Advanced Copier when planning inventory and promotions.
4. Use the monthly sales patterns to plan inventory, staffing, and marketing activities during periods when sales are expected to be higher.
5. The forecasting model can be improved in the future by adding more useful variables such as customer segment, discount, shipping mode, product category, and previous months' sales.

8. Conclusion

The Sales Performance & Forecasting Analysis identified important patterns in the Superstore dataset. Technology was the highest-performing category, while the West was the strongest region. Sales also showed substantial variation across months, with November 2018 recording the highest monthly sales.

The Random Forest model performed better than the initial Linear Regression model. The final Random Forest model achieved an R^2 score of 0.5511 and an RMSE of 16,094.10. The results show that adding time and seasonal information helped the model predict monthly sales better.

The model could be improved in the future by adding more useful features such as discounts, customer segments, product information, and previous months' sales.

9. GitHub Repository

The complete project files, Jupyter Notebook, dataset, charts, and final report are available on GitHub:

<https://github.com/ankita7833/Sales-Performance-Forecasting/tree/main>